Pet Breed Classifier

Architecture Design Document | Streamlit app + fine-tuned ResNet50

Item	Detail
Task	Single-label image classification: 37 cat and dog breeds (12 cats, 25 dogs)
Dataset	Oxford-IIIT Pet - 7,390 images, split 5,912 train / 739 validation / 739 test (stratified)
Model	ResNet50 (ImageNet) + GAP + Dense(256) + Dropout(0.5) + Dense(37, softmax); 24.1M parameters
Input	RGB image resized to 180 x 180
Result	89.0% test accuracy, 0.351 test loss
Frontend	Streamlit (upload or camera input, top-5 chart, low-confidence warning)
Packaging	Python 3.11, requirements.txt, Dockerfile, Git LFS for the model

1. Purpose and scope

This document describes how the trained notebook model is packaged into an interactive web application. The app lets a user upload or capture a photo and returns the predicted breed, its species, the model's confidence and the five most likely breeds. Training is performed offline in the notebook (`notebooks/oxford_III_cat_dog.ipynb`); the app only performs inference.

2. Design goals

- **Faithful preprocessing** - inference uses exactly the same resize and scaling as the training pipeline.
- **Simple deployment** - one process, no database, no external API; works locally, in Docker or on Streamlit Community Cloud.
- **Honest outputs** - show probabilities, not only a label, and warn when confidence is low.
- **Cached model** - load once per server process with `st.cache_resource`.

3. System architecture

The system is a single Python process. The browser talks to Streamlit; Streamlit calls small, testable modules for preprocessing, inference and label handling.

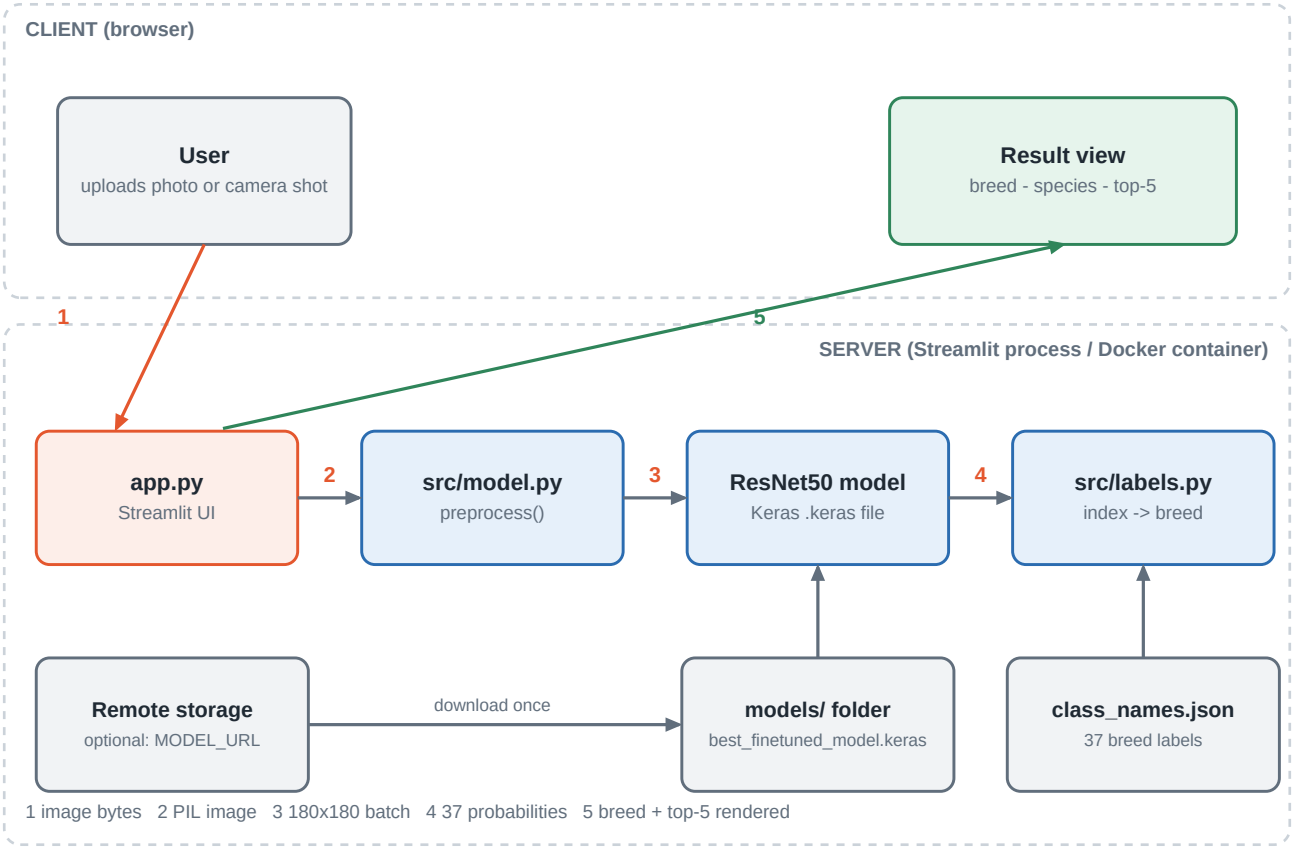


Figure 1 - Components and request flow

Components

Component	Responsibility
`app.py`	UI layout, file/camera input, caching, error handling, result rendering
`src/model.py`	Model loading (with Lambda custom object), preprocessing, top-k prediction, optional model download
`src/labels.py`	Loads class names, enforces model output order, species detection, display names
`models/`	*best_finetuned_model.keras* (~165 MB, Git LFS) and *class_names.json*
`tests/`	Unit tests for label logic (run without TensorFlow)

Streamlit execution model

Streamlit reruns the script on every interaction. Expensive objects are therefore cached: the model with `@st.cache_resource` (shared across sessions) and the class list with `@st.cache_data`.

4. Model architecture

A ResNet50 convolutional base pre-trained on ImageNet is reused as a feature extractor and partially fine-tuned. Preprocessing is embedded in the model, so callers only supply values in the 0-1 range.

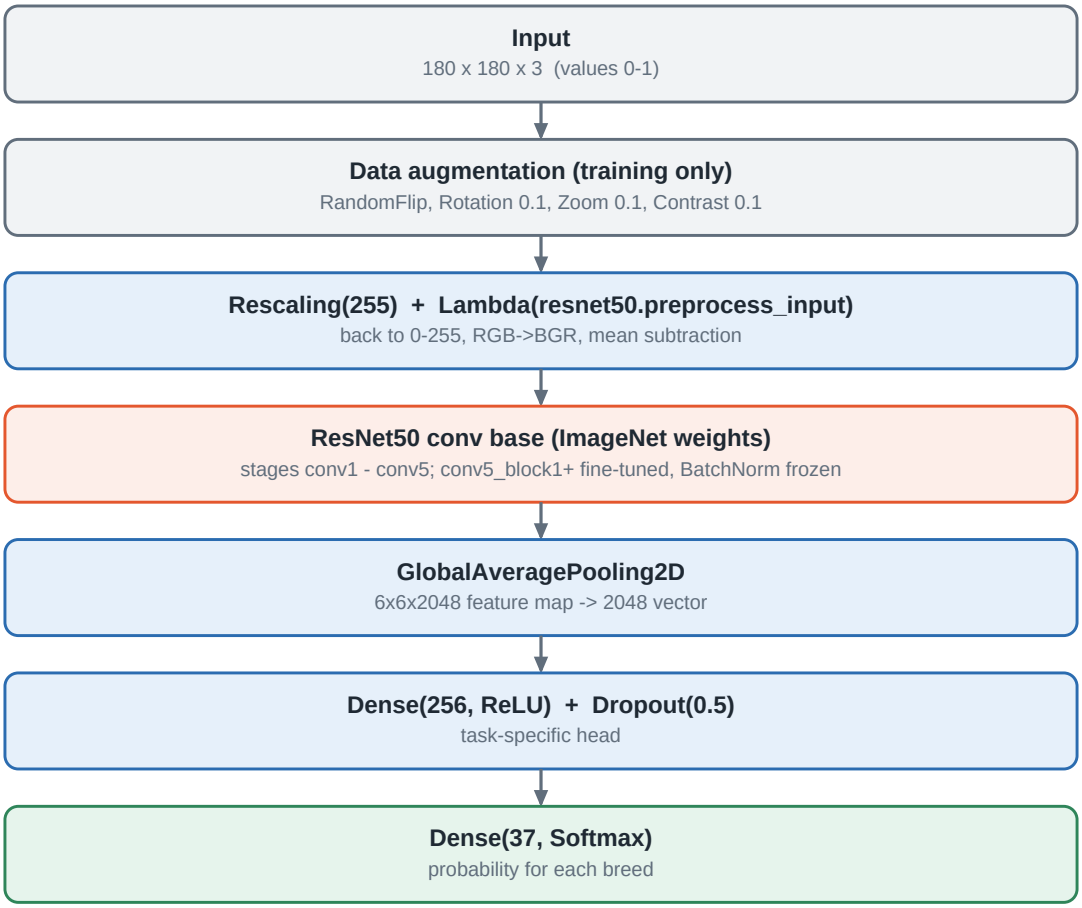


Figure 2 - Layer stack of *best_finetuned_model.keras*

Key design decisions

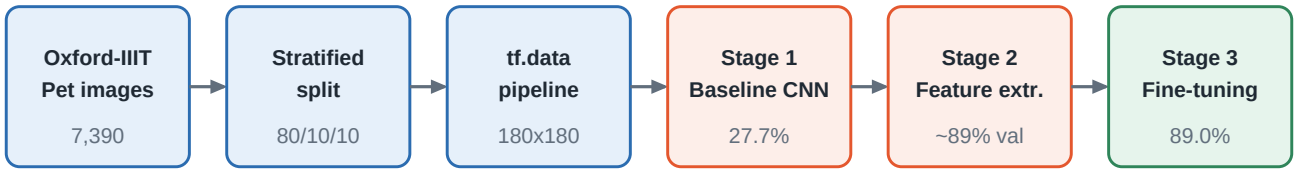
- **Transfer learning** - a CNN trained from scratch reached only 27.7% test accuracy; ResNet50 features lift this to 89%.
- **Partial fine-tuning** - layers from `conv5_block1_out` onward were unfrozen with a very low learning rate (1e-6). BatchNorm layers stay frozen to protect pre-trained statistics.
- **Embedded preprocessing** - Rescaling(255) followed by `preprocess_input` keeps the data pipeline simple (0-1 floats) but means the Lambda layer needs `custom_objects` and `safe_mode=False` at load time.
- **Regularisation** - augmentation (training only) and Dropout(0.5) in the head.

Output label mapping

Training labels were created from `sorted(unique_breeds)`, so output neuron *i* maps to the *i*-th breed in alphabetical order (uppercase cat breeds first, e.g. 0 = Abyssinian, 1 = Bengal, ..., 12 = american_bulldog). The notebook exported `class_names.json` in dataset-shuffle order instead; `labels.py` always re-sorts the names, which restores the correct mapping.

5. Training pipeline

Training happens offline in Google Colab. The exported model is the artefact consumed by the app.



test accuracy shown for stages 1 and 3; stage 2 shows validation accuracy

Figure 3 - Training stages and results

Stage	Setup	Optimizer / LR	Result
1. Baseline CNN	4 x Conv-Pool, Dense(256), Dropout; 5.7M params, from scratch	Adam 1e-3	27.7% test acc
2. Feature extraction	ResNet50 frozen, new head trained, up to 15 epochs	Adam 1e-3	~89% val acc
3. Fine-tuning	Unfreeze conv5_block1_out onward (BN frozen), up to 10 epochs	Adam 1e-6	89.5% val, 89.0% test

Data pipeline

- Labels come from file names (`leonberger_58.jpg` -> *leonberger*); 37 balanced classes of about 200 images each.
- Shuffle (seed 42), then 80% train and a 50/50 split of the remainder into validation and test, stratified by class.
- `tf.data`: read file, decode JPEG (3 channels), bilinear resize to 180x180, divide by 255, batch 32, prefetch.
- EarlyStopping on validation loss (patience 5, restore best weights).

6. Inference sequence

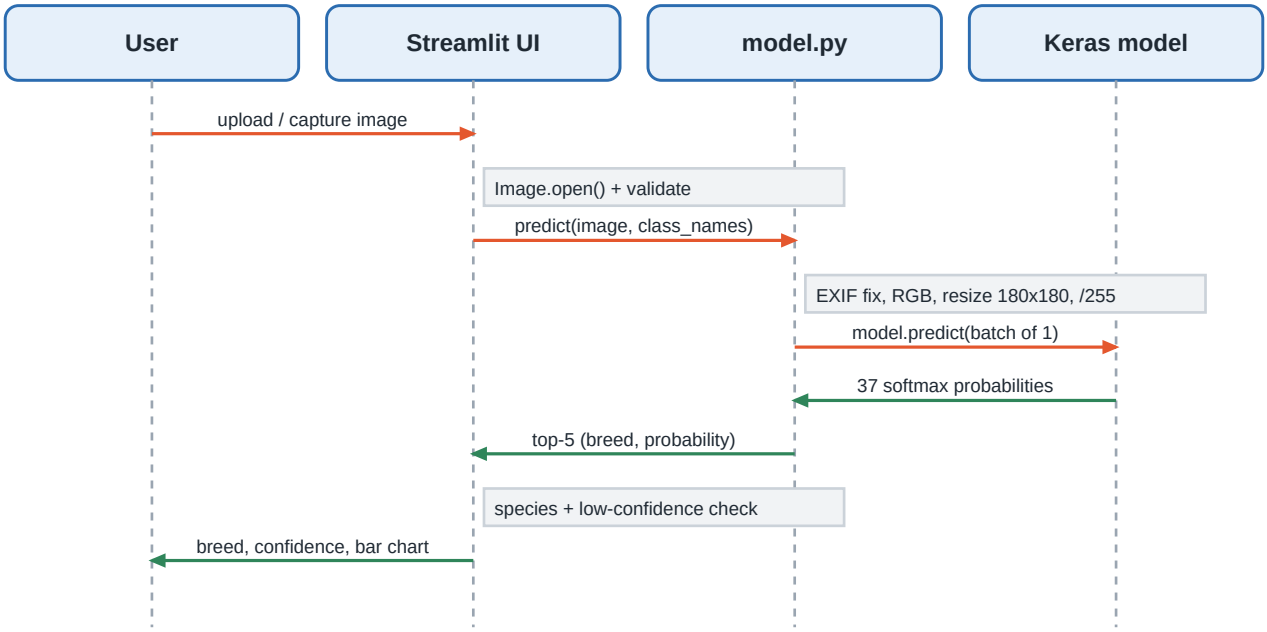


Figure 4 - Single prediction

7. Deployment

Option	How	Notes
Local	`pip install -r requirements.txt` `streamlit run app.py`	Fastest way to try; needs the model in models/
Docker	`docker build -t pet-app .` `docker run -p 8501:8501 pet-app`	Uses tensorflow-cpu; reproducible
Streamlit Community Cloud	Connect GitHub repo, main file app.py	Git LFS files are fetched on deploy; or set MODEL_URL secret
Hugging Face Spaces	Streamlit/Docker Space; host the model in the Hub	Good option for files > 100 MB

Model hosting

GitHub rejects single files above 100 MB, and the model is about 165 MB. Two supported approaches: (a) **Git LFS** (`.gitattributes` already tracks *.keras; free quota is 1 GB storage / 1 GB bandwidth per month), or (b) host the file elsewhere (GitHub Release asset, Hugging Face Hub, cloud bucket) and set the `MODEL_URL` environment variable; the app downloads it once on first start.

Configuration

Name	Where	Purpose
MODEL_URL	env var or .streamlit/secrets.toml	Direct-download URL used when models/best_finetuned_model.keras is missing
maxUploadSize	.streamlit/config.toml	Upload limit in MB (default 10)

8. Repository layout

```
app.py          Streamlit UI
src/            model.py, labels.py
models/         best_finetuned_model.keras (LFS), class_names.json
notebooks/      training notebook
docs/           architecture.pdf + generator script
tests/          pytest unit tests
Dockerfile, requirements.txt, .gitignore, .gitattributes, ruff.toml, LICENSE, README.md
.github/workflows/ci.yml  lint + tests
```

9. Limitations and future work

- **Closed set** - only 37 breeds. Softmax always produces a breed, so other animals or objects get a confident-looking wrong answer; the app only warns below 50% confidence. Adding an 'other / not a pet' class or an out-of-distribution detector would help.
- **Mixed breeds** - the dataset contains purebreds; mixed breeds will be mapped to the nearest known breed.
- **Accuracy** - 89% overall; visually similar pairs (e.g. American Bulldog vs. Staffordshire Bull Terrier, Ragdoll vs. Birman) are the main errors. Higher resolution (224+), longer fine-tuning or a newer backbone (EfficientNetV2, ConvNeXt) are natural upgrades.
- **Model security** - Lambda layers require `safe_mode=False`; only load model files you produced. Re-exporting the model without a Lambda layer (preprocessing in the app) would remove this requirement.
- **Throughput** - CPU inference of one image takes well under a second, which is fine for a demo; add batching or a model server for heavy traffic.